



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	MD Razia
Student Name	Ritwik
Roll Number	2520080051
Branch	AI&DS
Course Outcome	Co-5

SortBench – Empirical Evaluation of Advanced Sorting Algorithms

Work Done Summary:

The SortBench case study evaluated and implemented five advanced sorting algorithms, Merge Sort, Quick Sort (with three pivot strategies), Heap Sort, Counting Sort, and Radix Sort, and benchmarked their performance empirically on simulated flow-rate sensor-log datasets of varying size and shape (random, sorted, reverse-sorted, and nearly-sorted). Each algorithm was timed across multiple dataset sizes, checked for output correctness, tested for stability using tagged records, and profiled for peak auxiliary memory usage. The benchmarks revealed that Quick Sort using a fixed last-element pivot degraded to its $O(n^2)$ worst case on already-sorted input, running over 270 times slower than every other algorithm at 10,000 elements, while randomized and median-of-three pivot strategies avoided this degradation entirely. Counting Sort and Radix Sort consistently outperformed all comparison-based sorts in raw speed for the bounded-integer dataset used, at the cost of additional memory proportional to the value range or digit count.

Implementation Details:

1. Implemented Merge Sort using a recursive divide-and-conquer approach with auxiliary arrays for merging sorted halves.
2. Implemented Quick Sort with three interchangeable pivot-selection strategies: last-element pivot, randomized pivot, and median-of-three pivot, using Lomuto partitioning.
3. Implemented Heap Sort by building a max-heap in place and repeatedly extracting the maximum element to the end of the array.
4. Implemented Counting Sort for non-negative integer datasets by counting occurrences of each value and reconstructing the sorted output.
5. Implemented Radix Sort (LSD, base 10) by applying a stable counting sort on each digit position from least significant to most significant.
6. Generated four dataset shapes (random, sorted, reverse-sorted, nearly-sorted) at sizes of 1,000, 5,000, and 10,000 elements to empirically benchmark execution time for every algorithm.

7. Verified algorithm correctness against Python's built-in sort, tested stability using tagged (key, original-index) records, and profiled peak auxiliary memory usage with tracemalloc.

Results and Screenshots:

Runtime Benchmark: Random vs. Sorted Datasets (Quick Sort Worst Case):

```
WATERGRID SENSOR-LOG SORTING BENCHMARK
Dataset: simulated flow-rate sensor readings (l/min)
=====

Dataset type: RANDOM
Algorithm                n=1000    n=10000
-----
Merge Sort                1.432ms   17.895ms
Quick Sort (Last Pivot)   0.790ms   12.055ms
Quick Sort (Random Pivot) 1.081ms   17.091ms
Quick Sort (Median-of-3 Pivot) 1.124ms  15.899ms
Heap Sort                 1.863ms   21.717ms
Counting Sort             1.309ms    1.424ms
Radix Sort                 0.595ms    5.833ms

Dataset type: SORTED (already in order)
Algorithm                n=1000    n=10000
-----
Merge Sort                1.003ms   11.376ms
Quick Sort (Last Pivot)   30.527ms  3154.834ms
Quick Sort (Random Pivot) 0.917ms   11.686ms
Quick Sort (Median-of-3 Pivot) 0.803ms  10.259ms
Heap Sort                 1.479ms   22.154ms
Counting Sort             0.146ms    1.382ms
Radix Sort                 0.395ms    5.622ms

Observation: Quick Sort with a fixed last-element pivot
degrades to its  $O(n^2)$  worst case on sorted input -
3.15 seconds at n=10000, ~270x slower than every other
algorithm. Random and median-of-3 pivots avoid this.
```

Stability Verification on Tagged Sensor Records:

```
WATERGRID SENSOR-LOG STABILITY VERIFICATION
=====

Input flow buckets (l/min), tagged with record index:
['50#0', '20#1', '50#2', '10#3', '20#4', '50#5',
 '10#6', '30#7', '20#8', '10#9']

Algorithm                Stable?
-----
Merge Sort                True
Quick Sort (Last Pivot)   False
Quick Sort (Random Pivot) False
Quick Sort (Median-of-3 Pivot) False
Heap Sort                 False
Counting Sort             True (by construction)
Radix Sort                 True (by construction)

Note: Counting Sort and Radix Sort are stable by
construction (LSD radix relies on this at every digit
pass). Merge Sort preserves tie order naturally; in-place
Quick Sort and Heap Sort do not.
```

Auxiliary (Extra) Memory Usage Profile:

```
WATERGRID AUXILIARY MEMORY PROFILE
(n=10000, random sensor-reading dataset)
=====

Algorithm                                Peak Extra Mem (KB)
-----
Merge Sort                               244.70
Quick Sort (Last Pivot)                   78.91
Quick Sort (Random Pivot)                 78.95
Quick Sort (Median-of-3 Pivot)            79.10
Heap Sort                                 78.69
Counting Sort                             332.64
Radix Sort                                156.85

Observation: Quick Sort and Heap Sort sort in-place,
using the least extra memory. Merge Sort needs auxiliary
arrays for merging. Counting Sort allocates a count array
sized to the value range, and Radix Sort allocates a
fresh output array on every digit pass.
```

Trade-off Comparison Table:

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?	Best Suited For
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Stable sorting of large datasets; external sorting
Quick Sort (last pivot)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	General use, but risky on sorted/structured data
Quick Sort (random/median-3)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ rare	$O(\log n)$	No	General-purpose, in-place sorting at scale
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Memory-constrained systems; guaranteed bound needed
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes	Small integer ranges (k small relative to n)
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes	Fixed-width integers/IDs; sensor or log timestamps

In this case study, we evaluated five advanced sorting algorithms on a common, empirically benchmarked footing rather than by theoretical complexity alone. Merge Sort offered the most predictable $O(n \log n)$ behaviour regardless of input shape, at the cost of higher auxiliary memory, making it well suited to stable sorting of large datasets. Quick Sort delivered the fastest practical runtimes on random data but proved highly sensitive to pivot choice: a fixed last-element pivot collapsed to quadratic time on sorted and

reverse-sorted input, while randomized and median-of-three pivots restored near-optimal performance, confirming that pivot strategy is not a minor implementation detail but a correctness-adjacent design decision. Heap Sort guaranteed $O(n \log n)$ performance in every case with the smallest memory footprint among the comparison-based sorts, but its lack of stability and weaker cache locality made it less attractive than Quick Sort when average-case speed mattered more than worst-case guarantees. Counting Sort and Radix Sort, both linear-time and stable by construction, dramatically outperformed every comparison-based algorithm on the bounded-integer dataset, but their applicability is limited to data with a small or fixed-width key range, such as sensor IDs, timestamps, or bucketed readings, rather than arbitrary comparable data.

GitHub Link: <https://github.com/ritwikxdevv/DSA-CASE-STUDIES>

Student Signature	Faculty Signature
-------------------	-------------------